



**ASSIGNMENTS FOR THE COURSE: PROJECT: COMPUTER
SCIENCE PROJECT (CSEMCSPCSP01)**

SUBMITTED BY - MOHAMMED SHAHEED SIDDIQUI

COURSE – MASTER’S IN COMPUTER SCIENCE

MATRICULATION.NO - 10244102

1.1.2 Development Phase

Abstract

This project aims to build a machine learning model to predict customer attrition in the telecom industry with the help of IBM Telco customer dataset. Multiple classification models were tested following pre-processing, encoding feature values, and using SMOTE to balance the data sets. Random Forest had the highest accuracy, a strong performance prediction model. It can be used to detect early signs of risk from at-risk customers and assists the telecom companies to identify the data-driven way to better manage customer retention.

Table of Contents

Introduction	4
Related Work.....	4
Technical Background.....	5
Methodology.....	6
Implementation	7
Testing.....	19
Conclusion.....	24
References	25
Appendix 1: Github Link:	27

Introduction

Customer churn is a major issue in the telecom service sector, as customers opt to terminate their service agreements and move to alternative providers, resulting in substantial revenue losses and missed market share. The competition grows and the problems of customer churn are calling for proactive identification of consumers that are likely to defect to specific actions that could enhance their retention (Chang, et al., 2024). The issue with this project is that the approach used previously fails to predict the churn behaviour in advance accurately. The study has made use of machine learning techniques to analyse the customer data and develop a predictive model in order to identify the high-risk customers. The aim is to make well-informed decisions and implement better customer retention strategies.

Related Work

The telecom industry is widely used, known, and studied for churn prediction because of its evident and direct effect on the company's revenue and customer retention strategies. Research in the early days, focused mainly on traditional statistics methodologies like logistic regression, and survival analysis, to gain insight into customer behavior. But such approaches were typically unable to handle "non-linear and more complex relationships [in] a large data set".

The improvements in the field of machine learning have led to an increased preference for classification methods like Decision Trees, Support Vector Machines and k-Nearest Neighbors for churn forecasting (Imani et al., 2025). Subsequent works incorporated ensemble learning methods, such as Random Forest and Gradient Boosting models, that boosted performance by combining several weak learners using either methods. An imbalanced dataset has been identified as a major challenge in previous studies, in which there are hundreds to thousands of non-churning customers for each of a set of customers that have churned. This imbalance frequently results in skewed models with poor performance

when learning minority classes. These researchers have tried to implement resampling procedures like SMOTE (Synthetic Minority Over-sampling Technique) to create synthetic examples for the minority class that help to learn the model.

The significance of feature selection in order to enhance the accuracy of churn prediction has been discussed in recent literature. Factors like contract types, tenure and monthly costs demonstrate a steady inclination to customer churn (Ribeiro et al., 2024). There remains a number of difficulties to overcome in order to obtain optimal generalization from the noisy actual data in the telecom and since customers' behavior may change with time properties (Alhaqui et al., 2024). This project extends prior developed work through the use of several ensemble models and SMOTE balancing techniques to enhance prediction accuracy and enable proactive customer retention measures in the telecommunications domain.

Technical Background

The development of this project is done in the Python programming language and under Google Colab, a cloud-based platform equipped for machine learning or data analysis tasks. It is implemented using commonly used data science libraries like Pandas and NumPy, which are suitable for the manipulation, cleaning and preprocessing of the data. Libraries aid in the efficient management of structured datasets, like addressing missing values and transforming features (Afkanpour et al., 2024). The Scikit-learn library is widely used for the development of machine learning models. It offers important tools for dividing the data into training and testing sets, transforming categorical data, and creating models, like Decision Tree classifiers, and Random Forest classifiers. Furthermore, gradient boosting techniques are implemented in the XGBoost library, extending its prediction capabilities by incrementally learning from these predictions.

The imbalanced-learn library of Python packages is used to solve the imbalance problem on the data set, using the SMOTE (Synthetic Minority Over-sampling Technique). This approach leads to good quality of the models by producing synthetic samples of the minor class. For model evaluation: Scikit learn metrics like the accuracy score, confusion matrix, precision, recall, and F1 score are used. Results are interpreted and visualized with libraries Matplotlib and Seaborn, which allow for easier understanding of the patterns and performance of a model (Lavanya et al., 2023). Borrowed from Google Colab, this is the environment in which the development takes place, with packages being and GPU being supported. Developed in Google Colab, no setup is required locally and packages can be easily installed.

Methodology

The process methodology of this project consists of 4 steps, including a machine learning pipeline for building a customer churn predictive model within the field of telecommunications. The first step is choosing a dataset and using the **IBM Watson Telco Customer Churn dataset** as the main data source (Kaggle, 2026). This data set includes demographic information about customers, use of services and customer-subscription information. During the data preprocessing, irrelevant features like customerID are dropped, quantitative features like TotalCharges are filled with the median value when the data is missing. Label encoding transforms a categorical variable into a numerical one, making it suitable for use in machine learning algorithms.

However, in the existing dataset, there is a severe imbalance problem between the majority and the minority classes, where the minority class constitutes churn customers. The existing minority class is sufficiently enriched by creating synthetic samples to face the imbalance situation for this dataset; this method is known as SMOTE (Synthetic Minority Over-sampling Technique) (Elreedy, D., Atiya & Kamalov, 2024). This can provide a well-

rounded learning as well as better model performance. The dataset is then divided into two sets: 80% for training and 20% for testing. Many machine learning classifiers such as **Decision Tree, Random Forest and XGBoost** are applied to the preprocessed data. The models are chosen in accordance with their capability of processing structured data and capturing intricate patterns. Finally, accuracy score, confusion matrix, precision, recall, and F1-score are computed for model evaluation and which model is performing best is assessed to predict churn.

Implementation

IMPORT LIBRARIES

```
[1]
✓ 18
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
import seaborn as sns
from imblearn.over_sampling import SMOTE

import warnings
warnings.filterwarnings("ignore")
```

Figure 1: Import Libraries

This is a graph for the importation of necessary Python libraries like Pandas, NumPy, Scikit-learn, Matplotlib, seaborn, XGBoost data etc. These libraries enable data preparation, data visualization, building machine learning models with them and evaluation. The class imbalance in the data is also addressed using SMOTE.

LOAD DATASET

[2]

✓ Os

```
Customer_Churn = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
```

```
print("Dataset Shape:", Customer_Churn.shape)
```

```
print(Customer_Churn.head())
```

✓

Dataset Shape: (7043, 21)

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	\
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GMVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	
4	9237-HQITU	Female	0	No	No	2	Yes	

	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	\
0	No phone service	DSL	No	...	No	
1	No	DSL	Yes	...	Yes	
2	No	DSL	Yes	...	No	
3	No phone service	DSL	Yes	...	Yes	
4	No	Fiber optic	No	...	No	

	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	\
0	No	No	No	Month-to-month	Yes	
1	No	No	No	One year	No	
2	No	No	No	Month-to-month	Yes	
3	Yes	No	No	One year	No	
4	No	No	No	Month-to-month	Yes	

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No
1	Mailed check	56.95	1889.5	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes

[5 rows x 21 columns]

Figure 2: Load Dataset

This figure shows populating the IBM Telco Customer Churn dataset in a dataframe, called "Customer_Churn". It displays the shape of the dataset (shows 7043 rows and 21 columns) and preview of first 5 row(s) confirming successful data import and understanding of dataset shape.

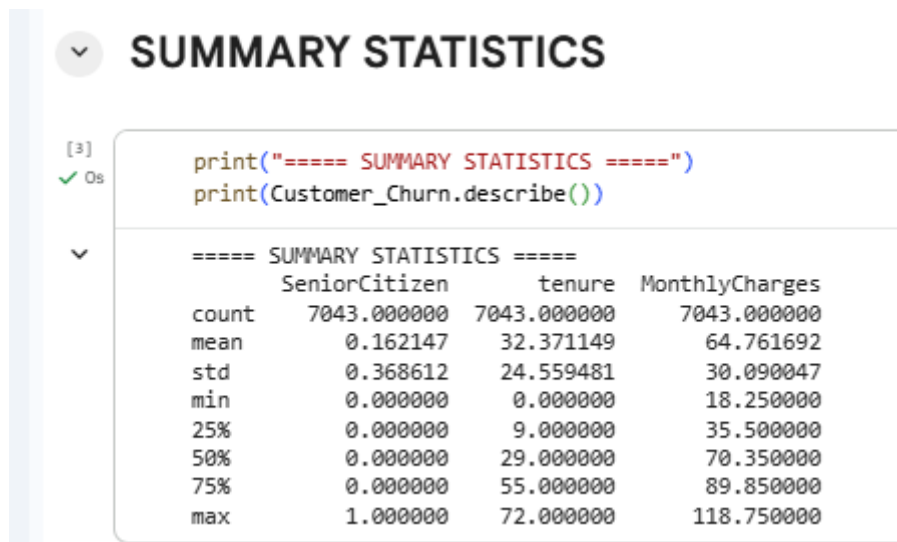


Figure 3: Summary Statistics

These numbers give some descriptive statistics for numerical variables like tenure, MonthlyCharges, and SeniorCitizen. The standard deviation, minimum and maximum, and quartile values are included, providing information about data distribution and variability.



Figure 4: Missing Values Analysis

This value is used to check whether the dataset is missing any value before and after preprocessing is done. It can not only guarantee that the data is complete, but also that it is clean and appropriate for machine learning modeling.

EDA VISUALIZATIONS

```
[5] sns.set(style="whitegrid")
```

CHURN DISTRIBUTION

```
[6] plt.figure(figsize=(6,4))
sns.countplot(x="Churn", data=Customer_Churn)
plt.title("Churn Distribution")
plt.show()
```

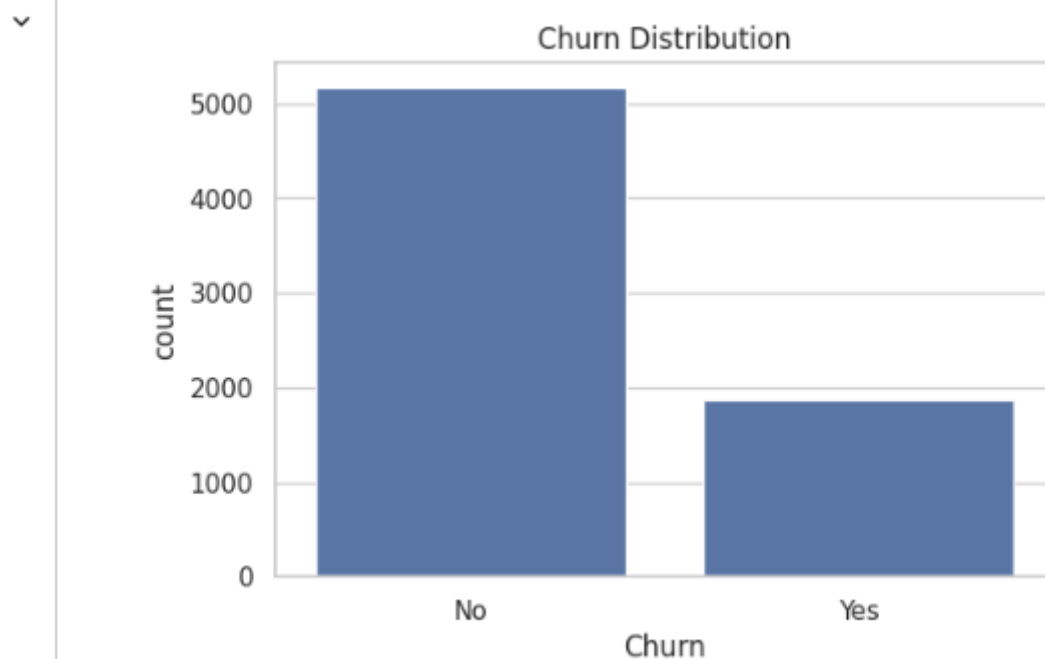


Figure 5: Churn Distribution

This number is a representation of the unevenness within the target variable. It indicates that, in fact, non-churn customers tend to be much higher than churn customers, so there is a need to implement the balancing techniques like SMOTE.

✓ CONTRACT TYPE vs CHURN

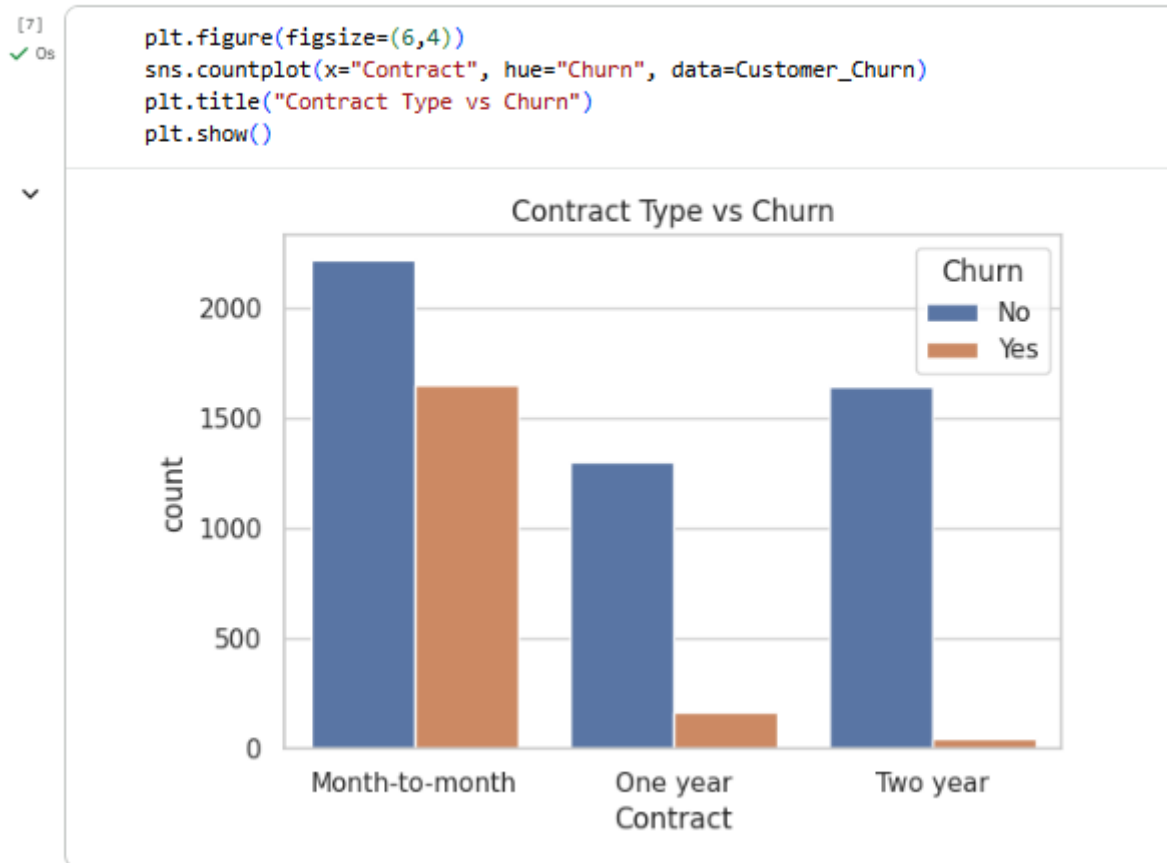


Figure 6: Contract Type vs Churn

This figure illustrates the link between churn behavior and type of contract.

Customers that have monthly contracts churn at the highest rate, leaving long term contracts (one-year and two years) lower churn, suggesting that contract duration has a significant impact on retention rates.

✓ TENURE DISTRIBUTION

```
[8]  
✓ Os  
plt.figure(figsize=(7,4))  
sns.histplot(Customer_Churn["tenure"], bins=30, kde=True)  
plt.title("Tenure Distribution")  
plt.show()
```

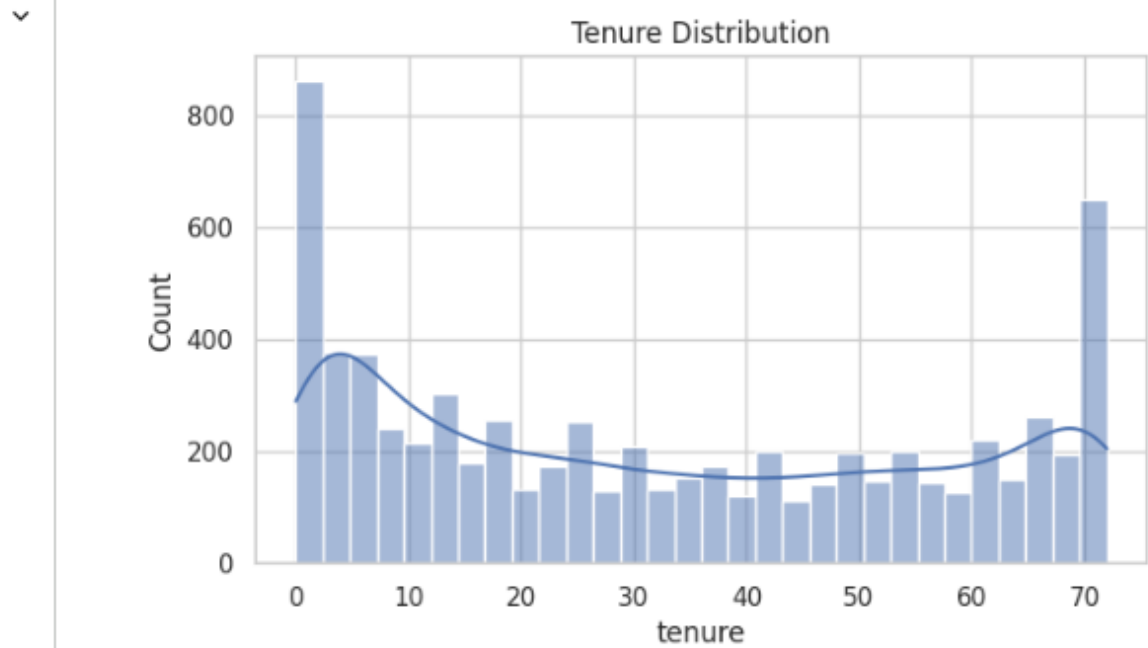


Figure 7: Tenure Distribution

This is a representation of the repeats of this data set. Customers are primarily new or long term, higher churn towards new customers, while the long-term customers churned have been around for a long time.

MONTHLY CHARGES vs CHURN

[9]
✓ Os

```
plt.figure(figsize=(7,4))  
sns.boxplot(x="Churn", y="MonthlyCharges", data=Customer_Churn)  
plt.title("Monthly Charges vs Churn")  
plt.show()
```

✓

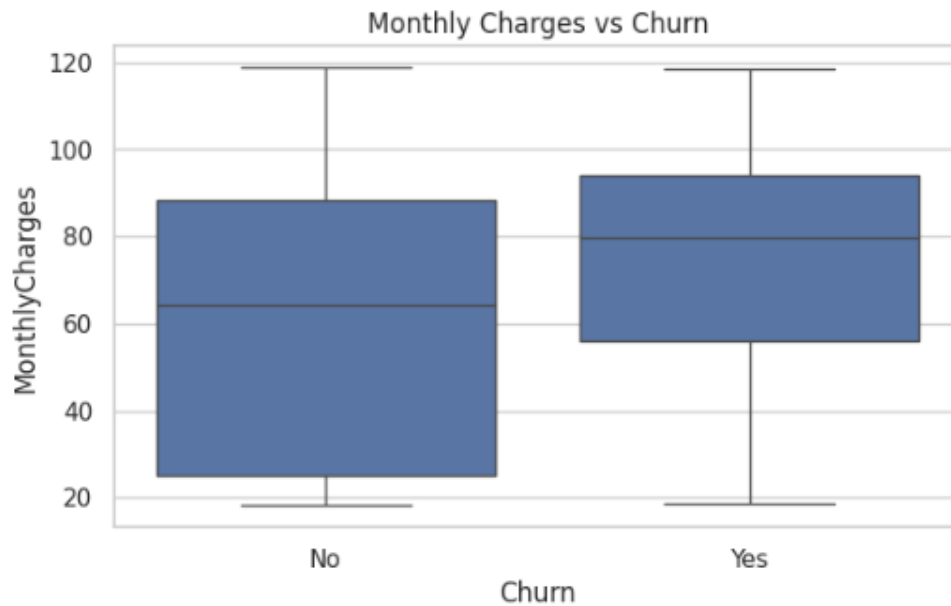


Figure 8: Monthly Charges vs Churn

This box plot shows the monthly charges of churn and non churn customers. It indicates that those who pay their bills with higher monthly amounts can be more likely to cancel their service, which means pricing is important for retaining customers.

INTERNET SERVICE IMPACT

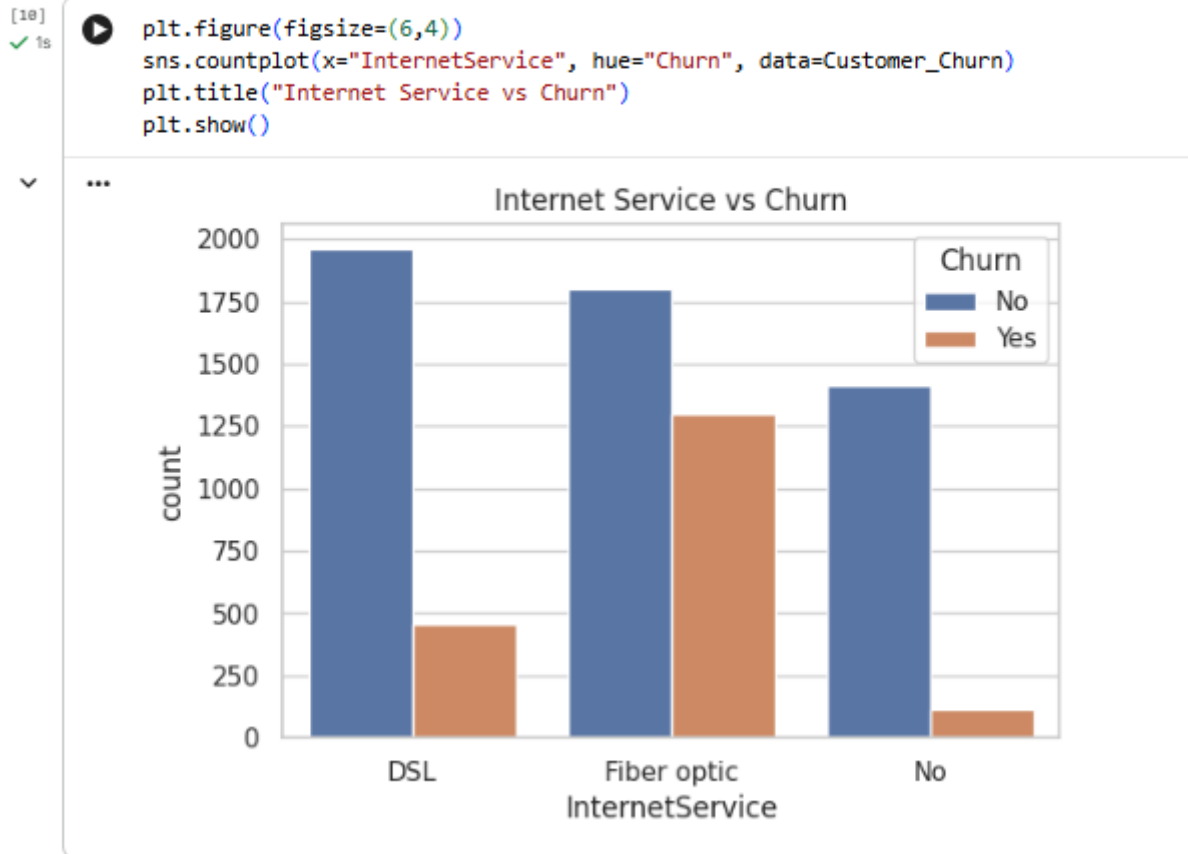


Figure 9: Internet Service vs Churn

This is charted based on internet services and churn. The big drop in user churn between fiber optic and DSL users reveals that service type is a significant factor in user churn behavior.

DATA CLEANING

```
[11]
✓ 0s # Remove customerID (not useful)
Customer_Churn.drop("customerID", axis=1, inplace=True)

[12]
✓ 0s # Convert TotalCharges to numeric
Customer_Churn["TotalCharges"] = pd.to_numeric(Customer_Churn["TotalCharges"], errors="coerce")

[13]
✓ 0s # Fill missing values
Customer_Churn["TotalCharges"].fillna(Customer_Churn["TotalCharges"].median(), inplace=True)
```

Figure 10: Data Cleaning

Data cleaning operations include removing the customerID field (as it doesn't add value), converting to a numeric data type and imputing missing values by the MEDIAN method. This can help ensure that the datasets are consistent and that their integrity is increased before using machine learning algorithms.

✓ LABEL ENCODING

[14]
✓ Os

```
le = LabelEncoder()

for column in Customer_Churn.columns:
    if Customer_Churn[column].dtype == "object":
        Customer_Churn[column] = le.fit_transform(Customer_Churn[column])

print(Customer_Churn.head())
```

✓

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	\
0	0	0	1	0	1	0	
1	1	0	0	0	34	1	
2	1	0	0	0	2	1	
3	1	0	0	0	45	0	
4	0	0	0	0	2	1	

	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	\
0	1	0	0	2	
1	0	0	2	0	
2	0	0	2	2	
3	1	0	2	0	
4	0	1	0	0	

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	\
0	0	0	0	0	0	
1	2	0	0	0	1	
2	0	0	0	0	0	
3	2	2	0	0	1	
4	0	0	0	0	0	

	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	1	2	29.85	29.85	0
1	0	3	56.95	1889.50	0
2	1	3	53.85	108.15	1
3	0	0	42.30	1840.75	0
4	1	2	70.70	151.65	1

Figure 11: Label Encoding

Label encoding is a process that takes place using LabelEncoder for converting categorical variables to numeric form. All columns with an object type are converted to integers to ensure that they are properly processed by machine learning algorithms for model training and prediction.

✓ SPLIT DATA

```
[15]
✓ Os      X = Customer_Churn.drop("Churn", axis=1)
          y = Customer_Churn["Churn"]

          X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Figure 12: Split Data

Data is split into features (X) and the label (Churn). It is then divided into training set and the testing set, keeping the ratio as 80:20, to evaluate the model by testing on unseen data, and measure the model's performance accordingly.

✓ HANDLE CLASS IMBALANCE (SMOTE)

```
[16]
✓ Os      smote = SMOTE(random_state=42)

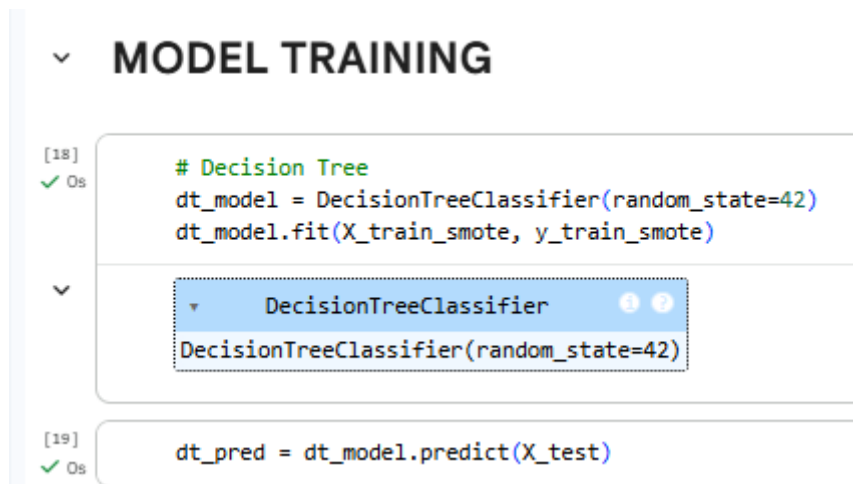
          X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

[17]
✓ Os      print("Before SMOTE:", y_train.value_counts())
          print("After SMOTE:", y_train_smote.value_counts())

          Before SMOTE: Churn
          0    4138
          1    1496
          Name: count, dtype: int64
          After SMOTE: Churn
          0    4138
          1    4138
          Name: count, dtype: int64
```

Figure 13: SMOTE Handling Class Imbalance

SMOTE is applied to balance out the dataset, synthetic minority class samples are generated. Data is imbalanced before being used in SMOTE. Following SMOTE, both classes are balanced, which enhances the fairness of the model and the prediction accuracy in detecting churn.



The image shows a Jupyter Notebook interface with a section titled "MODEL TRAINING". It contains two code cells. The first cell, labeled [18] and executed successfully (green checkmark, 0s), contains the following code:

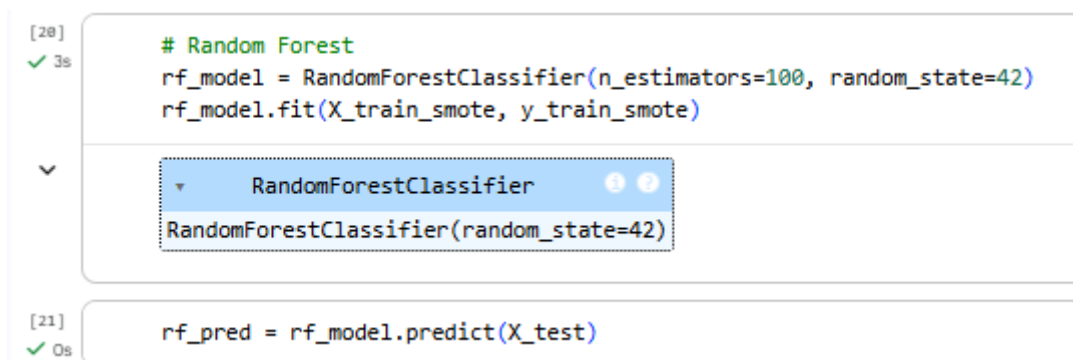
```
# Decision Tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train_smote, y_train_smote)
```

 Below the code, a variable inspector shows the object `DecisionTreeClassifier` with its attributes, including `DecisionTreeClassifier(random_state=42)`. The second cell, labeled [19] and also executed successfully (green checkmark, 0s), contains the code:

```
dt_pred = dt_model.predict(X_test)
```

Figure 14: Decision Tree Model Training

The training data for the Decision Tree classifier is SMOTE-balanced. It obtains rules for printing the classes of churn behavior based on features. Then the results of the prediction of test data outcomes are evaluated using the trained model.



The image shows a Jupyter Notebook interface with a section titled "MODEL TRAINING". It contains two code cells. The first cell, labeled [20] and executed successfully (green checkmark, 3s), contains the following code:

```
# Random Forest
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_smote, y_train_smote)
```

 Below the code, a variable inspector shows the object `RandomForestClassifier` with its attributes, including `RandomForestClassifier(random_state=42)`. The second cell, labeled [21] and also executed successfully (green checkmark, 0s), contains the code:

```
rf_pred = rf_model.predict(X_test)
```

Figure 15: Random Forest Model Training

To create stability in prediction and to prevent over-fitting 100 decision trees have been used in the Random Forest classifier. It combines together the predictions from more than one Tree to increase accuracy and robustness than a single Decision Tree model.

```
[22] ✓ 1s
# XGBoost
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
xgb_model.fit(X_train_smote, y_train_smote)

XGBClassifier(
    base_score=None, booster=None, callbacks=None,
    colsample_bylevel=None, colsample_bynode=None,
    colsample_bytree=None, device=None, early_stopping_rounds=None,
    enable_categorical=False, eval_metric='logloss',
    feature_types=None, feature_weights=None, gamma=None,
    grow_policy=None, importance_type=None,
    interaction_constraints=None, learning_rate=None, max_bin=None,
    max_cat_threshold=None, max_cat_to_onehot=None,
    max_delta_step=None, max_depth=None, max_leaves=None,
    min_child_weight=None, missing=nan, monotone_constraints=None,
    multi_strategy=None, n_estimators=None, n_jobs=None,
    num_parallel_tree=None, ...)

[23] ✓ 0s
xgb_pred = xgb_model.predict(X_test)
```

Figure 16: XGBoost Model Training

Gradient boosting technique is used to train a XGBoost classifier for optimizing the prediction performance. It sequentially fine tunes weak learners and reduces error with log loss function to achieve high accuracy in the tabular churn prediction dataset, which has structured chunches in the tabular format.

Testing

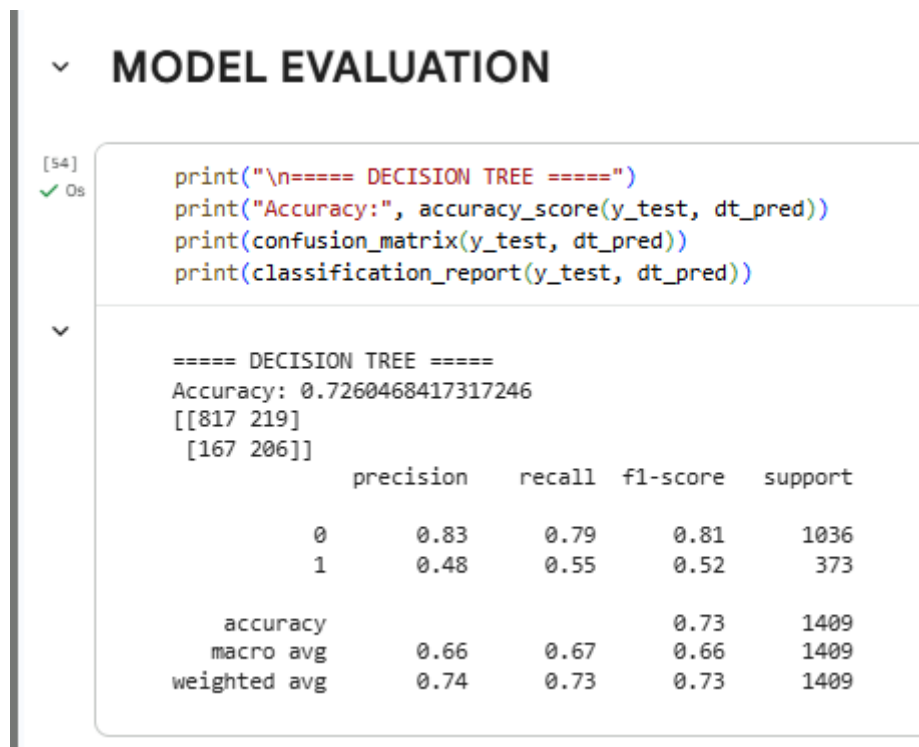


Figure 17: Decision Tree Model Evaluation

The Decision Tree model's accuracy is 72.6%. From the confusion matrix it can be seen that the overall performance is moderate, as there is better accuracy for non-churn class. But the lower recall among churn customers shows it is hard to identify customers that might churn from the telecom service.

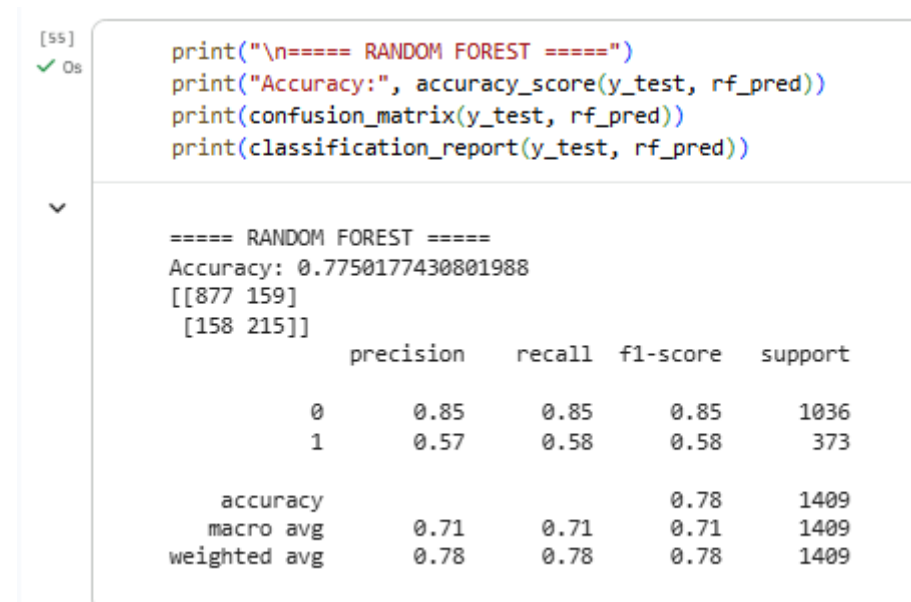
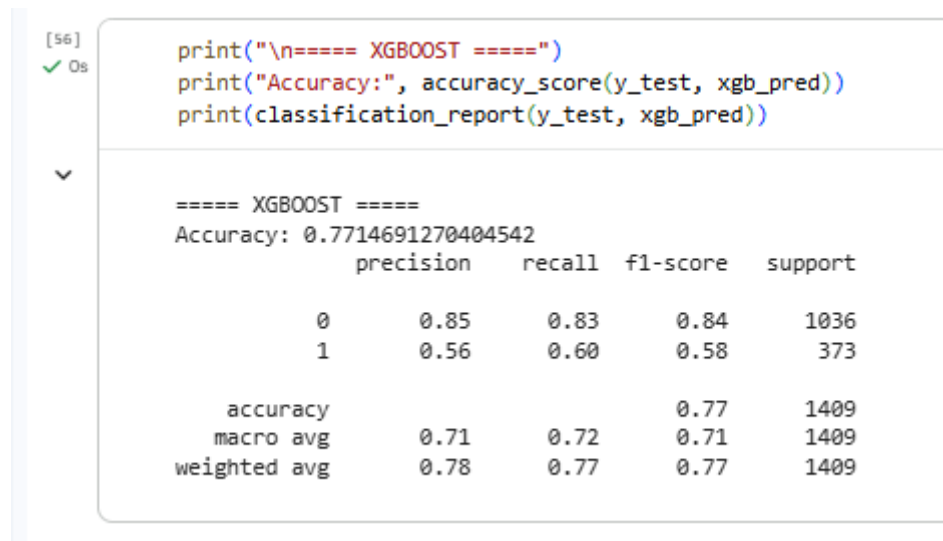


Figure 18: Random Forest Model Evaluation

Random Forest model performs better in terms of accuracy (77.5%) than Decision Tree. It gives a balanced precision and recall capabilities, particularly good churn detection. This means that there is an improvement in generalization and decrease in overfitting because of multiple decision trees involved in an ensemble learning.



```
[56] ✓ Os
print("\n===== XGBOOST =====")
print("Accuracy:", accuracy_score(y_test, xgb_pred))
print(classification_report(y_test, xgb_pred))
```

```
===== XGBOOST =====
Accuracy: 0.7714691270404542
```

	precision	recall	f1-score	support
0	0.85	0.83	0.84	1036
1	0.56	0.60	0.58	373
accuracy			0.77	1409
macro avg	0.71	0.72	0.71	1409
weighted avg	0.78	0.77	0.77	1409

Figure 19: XGBoost Model Evaluation

The accuracy of XGBoost is about 77% and very high on the precision of non-churn class. However, the recall for churn class is slightly lower than that of Random Forest, reflecting moderate performance in detecting churn customers (at risk) as compared to ensemble averaging techniques.

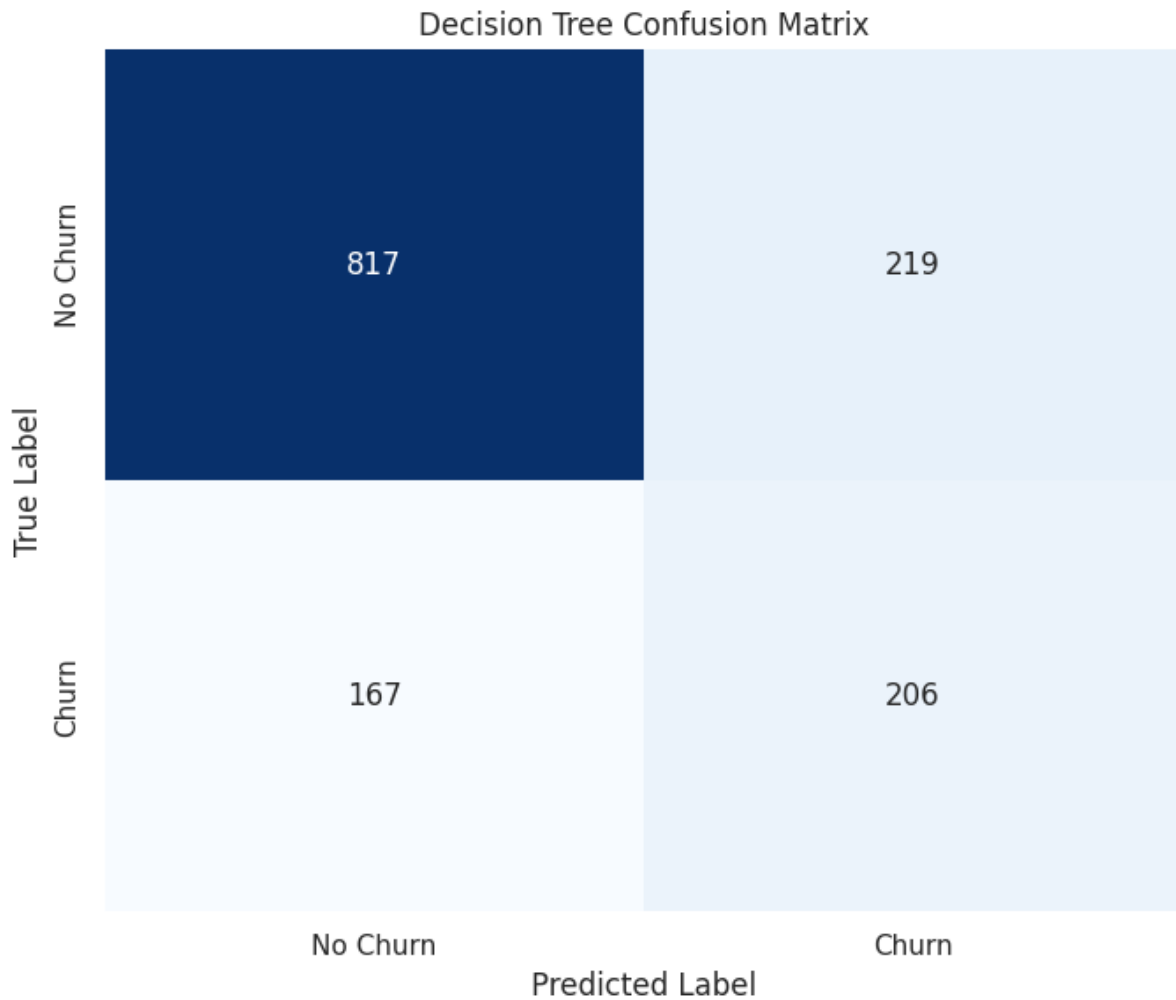


Figure 20: Decision Tree Confusion Matrix

The Decision Tree confusion matrix is good at correctly predicting both churn and non-churn classes with a moderate performance. But the number of mis-classifications is noteworthy, suggesting the poor predictive capacity as compared to Ensemble.

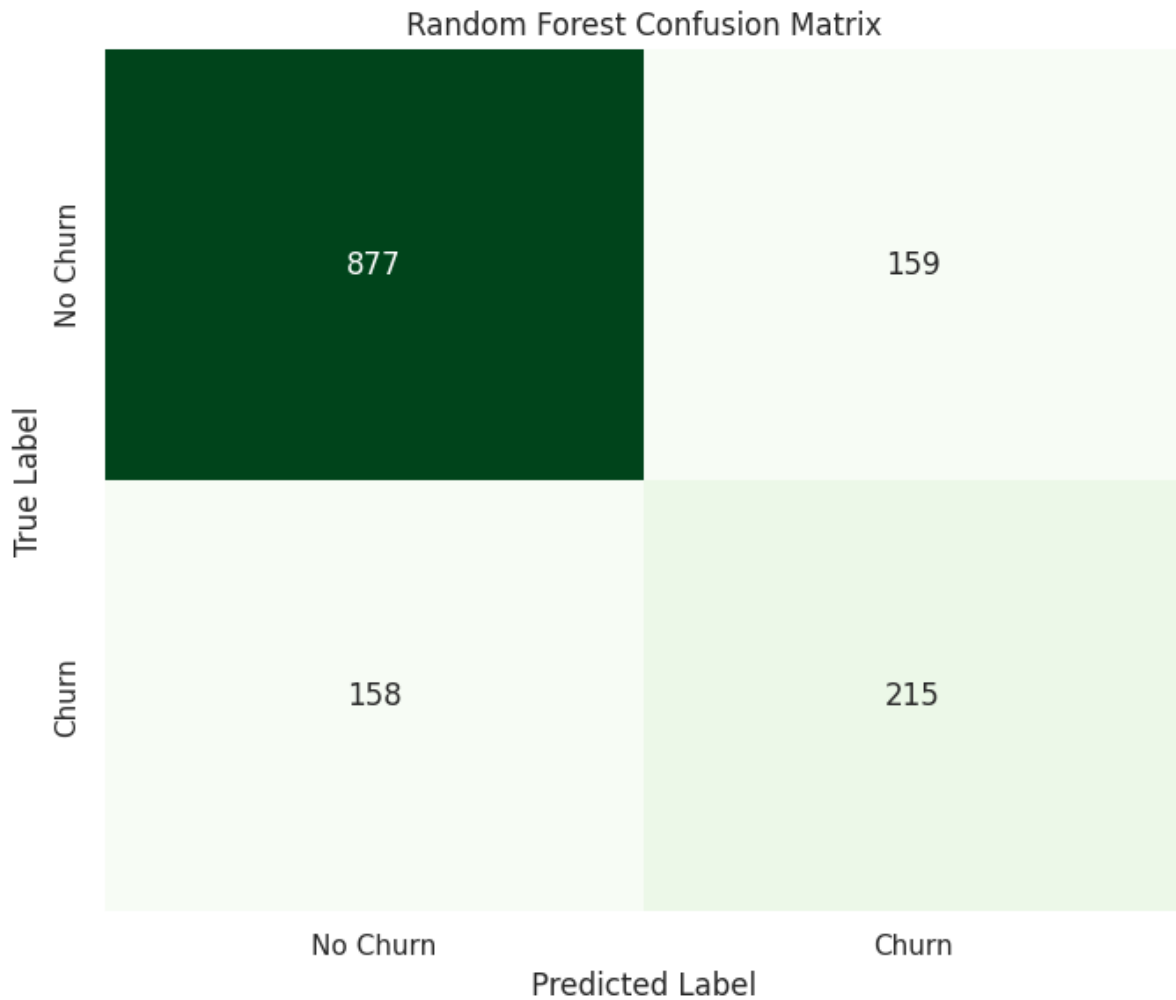


Figure 21: Random Forest Confusion Matrix

The random forest model displays better classification performance with greater accuracy in predicting both the classes. It decreases false positive and false negative and shows improved balance and better predictive power as compared to Decision Tree.

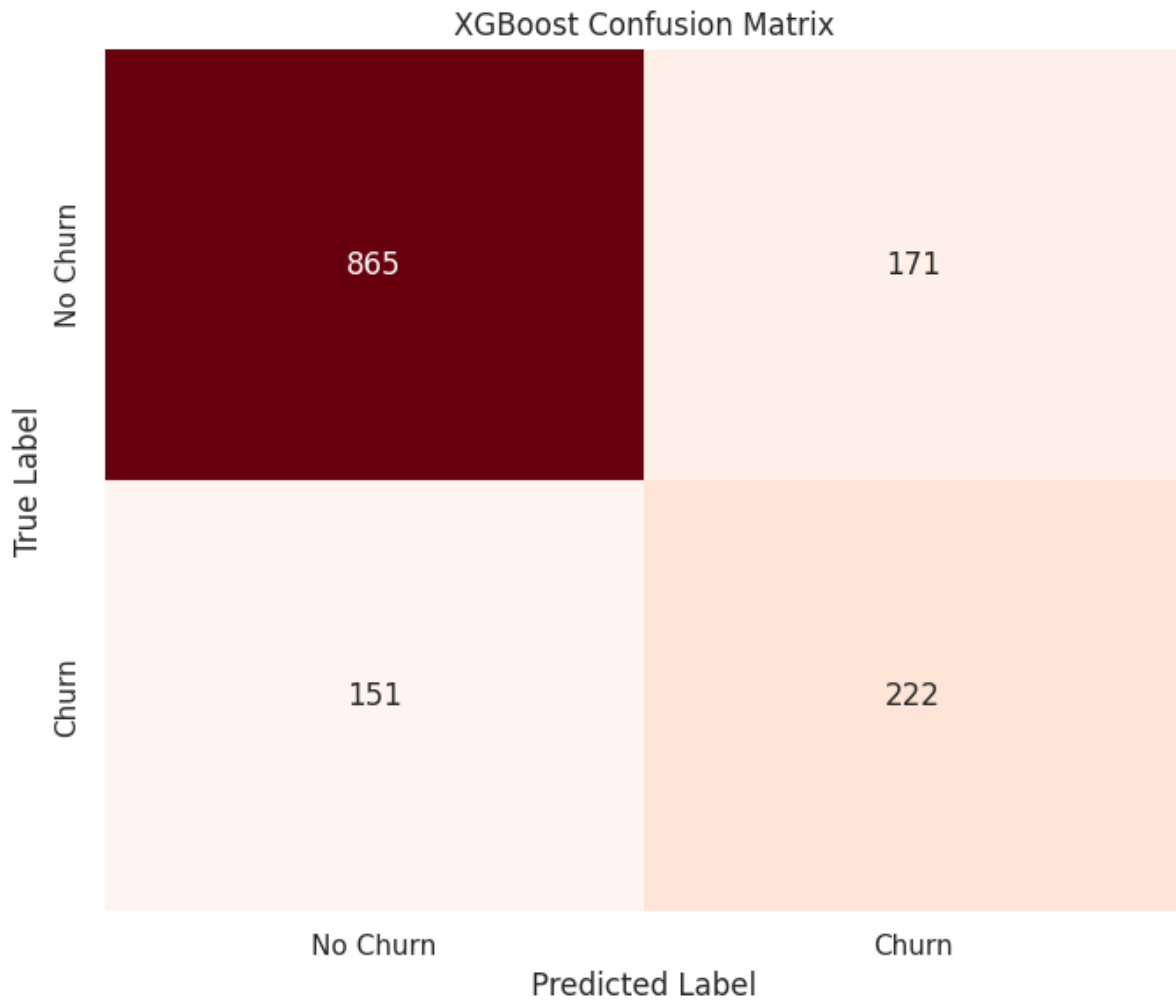


Figure 22: XGBoost Confusion Matrix

The XGBoost confusion matrix reveals a good performance in both churn detection and overall accuracy statistics when compared to Decision Tree. Overall, however, it is a reasonably good product with a level of misclassification, but the quality and quantity is fairly consistent.

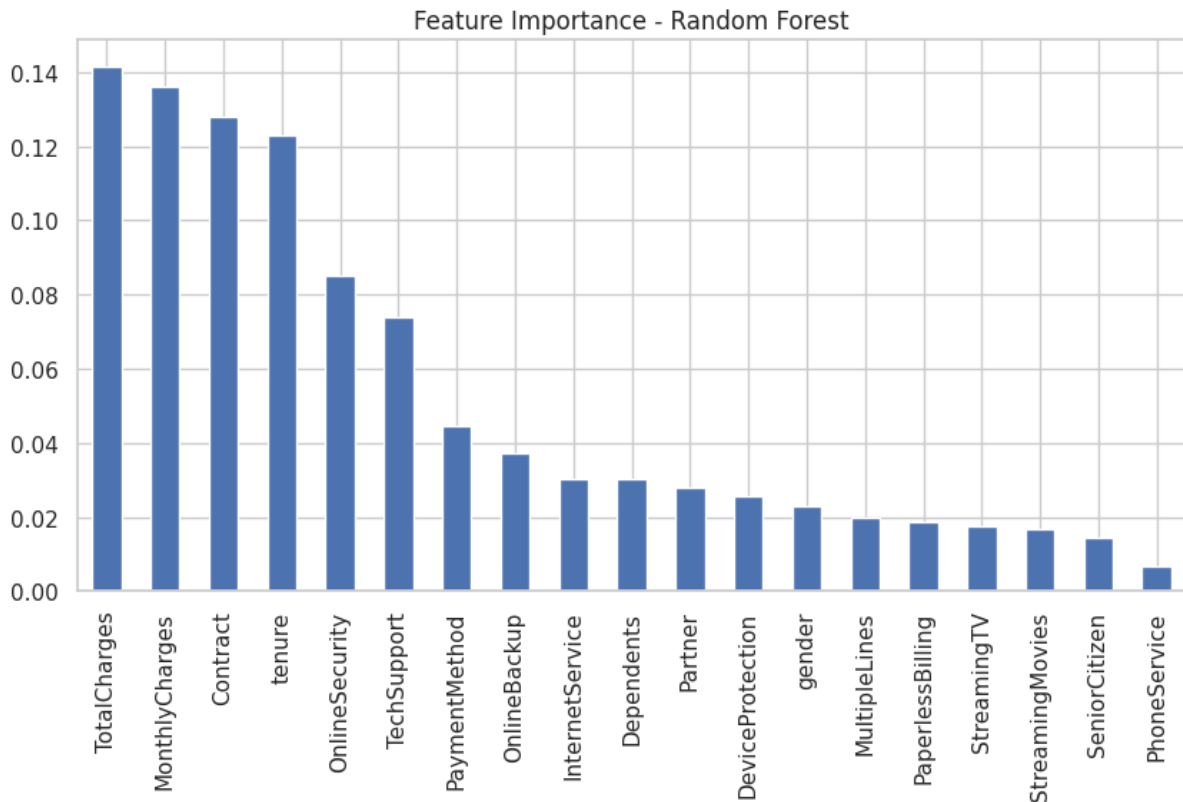


Figure 23: Feature Importance – Random Forest

Meaningful feature importances using Random Forest model. The most important predictors are TotalCharges, MonthlyCharges, Contract type and tenure. These functions have a significant effect on customer attrition and are crucial in terms of effective identification of at-risk customers.

Conclusion

This project was successful in the creation of a customer churn prediction system using machine learning approach in the telecommunication customer services. The study employed preprocessing techniques, SMOTE balancing to enhance the prediction accuracy and model performance using multiple classification models. The best to tackle the imbalanced and structured data was the Random Forest algorithm. The results show several factors that can cause churn, including contract type, length of contract, monthly charge. In conclusion, the model can be used by any telecom company to help them implement preventive customer retention practices and make informed decisions based on data.

References

- Afkanpour, M., Hosseinzadeh, E., & Tabesh, H. (2024). Identify the most appropriate imputation method for handling missing values in clinical structured datasets: a systematic review. *BMC Medical Research Methodology*, 24(1), 188.
<https://doi.org/10.1186/s12874-024-02310-6>
- Alhaqui, F., Lahsen-Cherif, I., Elkhechafi, M., & Elkhadimi, A. (2024). Deep learning for telecom self-optimized networks: Benefits and implications. *IEEE Access*, 12, 195422-195435. Digital Object Identifier 10.1109/ACCESS.2024.3521282
- Chang, V., Hall, K., Xu, Q. A., Amao, F. O., Ganatra, M. A., & Benson, V. (2024). Prediction of customer churn behavior in the telecommunication industry using machine learning models. *Algorithms*, 17(6), 231. <https://doi.org/10.3390/a17060231>
- Elreedy, D., Atiya, A. F., & Kamalov, F. (2024). A theoretical distribution analysis of synthetic minority oversampling technique (SMOTE) for imbalanced learning. *Machine Learning*, 113(7), 4903-4923. <https://doi.org/10.1007/s10994-022-06296-4>
- Imani, M., Joudaki, M., Beikmohammadi, A., & Arabnia, H. R. (2025). Customer churn prediction: A systematic review of recent advances, trends, and challenges in machine learning and deep learning. *Machine Learning and Knowledge Extraction*, 7(3), 105.
<https://doi.org/10.3390/make7030105>
- Kaggle (2026). *IBM Watson Telco Customer Churn*.
<https://www.kaggle.com/datasets/anandaraflii/ibm-watson-telco-customer-churn?resource=download>
- Lavanya, A., Gaurav, L., Sindhuja, S., Seam, H., Joydeep, M., Uppalapati, V., ... & Sagar, V. S. D. (2023). Assessing the performance of python data visualization libraries: a review. *Int. J. Comput. Eng. Res. Trends*, 10(1), 28-39.
<https://doi.org/10.22362/ijcert/2023/v10/i01/v10i0104>

Ribeiro, H., Barbosa, B., Moreira, A. C., & Rodrigues, R. G. (2024). Determinants of churn in telecommunication services: a systematic literature review. *Management review quarterly*, 74(3), 1327-1364. <https://doi.org/10.1007/s11301-023-00335-7>

Appendix 1: Github Link:

https://github.com/shaheed0123/MOHAMMED-SHAHEED-SIDDIQUI_CSEMCSPCSP01_-PCS-_P2_S.git